

# Sistema de Gestión - Intercoop

Sistema Full-Stack diseñado para la gestión y seguimiento de solicitudes internas. El proyecto ha sido desarrollado bajo una arquitectura orientada a capas, integrando protocolos de autenticación seguros y un diseño de interfaz responsivo.

## Tecnologías Utilizadas

- Backend: .NET 8 (C#) con Entity Framework Core.
- Base de Datos: PostgreSQL.
- Frontend: JavaScript nativo (Vanilla JS) y Tailwind CSS.
- Seguridad: JWT (JSON Web Tokens).

## Instrucciones para la puesta en marcha

Para ejecutar el proyecto en su entorno local, por favor siga los pasos descritos a continuación:

### 1. Requisitos Previos

- .NET 8 SDK instalado.
- PostgreSQL instalado y en ejecución.
- Editor de código (se recomienda Visual Studio Code).

### 2. Configuración de la Base de Datos

1. Cree una base de datos en su instancia de PostgreSQL denominada IntercoopDB.
2. Ejecute el script SQL proporcionado en la carpeta /scripts del repositorio para realizar la creación de las tablas y la definición de las relaciones de integridad.
3. Localice el archivo appsettings.json en la carpeta IntercoopAPI/ y actualice la cadena de conexión con los parámetros de su entorno local:

## JSON

```
"ConnectionStrings": {
```

```
  "DefaultConnection":
```

```
    "Host=localhost;Database=IntercoopDB;Username=SU_USUARIO;Password=SU_CONTRASEÑA"
```

}

### 3. Ejecución del Backend

1. Abra una terminal en el directorio IntercoopAPI/.
2. Restaure las dependencias del proyecto:

Bash

dotnet restore

3. Inicie el servidor de la API:

Bash

dotnet run

*El servicio estará disponible de manera predeterminada en el puerto 5284.*

### 4. Ejecución del Frontend

1. Diríjase a la carpeta Frontend/.
2. Abra el archivo index.html en su navegador de preferencia. Se recomienda utilizar el servidor local de su editor de código para asegurar la correcta comunicación con la API.

### Consideraciones Técnicas

- Eliminación Lógica: El sistema implementa una política de eliminación lógica en todos sus módulos. La visibilidad de los registros en la interfaz depende del valor booleano en la columna Activo dentro de la base de datos.
- Seguridad: Se utiliza autenticación basada en tokens JWT. Es imperativo que el archivo de configuración (appsettings.json) no sea compartido públicamente si contiene credenciales de acceso reales.

### Nota final

Este sistema ha sido desarrollado bajo los estándares solicitados en la prueba técnica, garantizando la escalabilidad del backend y la eficiencia en la gestión de datos.

# Documentación Técnica: Sistema de Gestión INTERCOOP

## 1. Arquitectura de Base de Datos (PostgreSQL)

El sistema utiliza una base de datos relacional normalizada para garantizar la integridad de la información. Todas las tablas implementan el patrón de **Eliminación Lógica** mediante el campo Activo (Booleano), lo que permite auditar y recuperar registros sin perder el historial.

### Script de BD.

```
-- BORRAMOS LAS MINÚSCULAS

DROP TABLE IF EXISTS "Solicitudes", "Usuarios", "Categorias", "Roles" CASCADE;

DROP TABLE IF EXISTS solicitudes, usuarios, categorias, roles CASCADE;

-- CREAMOS CON MAYÚSCULAS EXACTAS PARA C#

CREATE TABLE "Roles" (
    "IdRol" SERIAL PRIMARY KEY,
    "Nombre" VARCHAR(50) NOT NULL UNIQUE,
    "Activo" BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE "Categorias" (
    "IdCategoria" SERIAL PRIMARY KEY,
    "Nombre" VARCHAR(100) NOT NULL UNIQUE,
    "Descripcion" VARCHAR(255) NULL,
    "FechaCreacion" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "Activo" BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE "Usuarios" (
    "IdUsuario" SERIAL PRIMARY KEY,
    "Nombre" VARCHAR(100) NOT NULL,
    "Correo" VARCHAR(150) NOT NULL UNIQUE,
    "PasswordHash" VARCHAR(255) NOT NULL,
    "IdRol" INT NOT NULL,
    "FechaCreacion" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    "Activo" BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT fk_usuario_rol FOREIGN KEY ("IdRol") REFERENCES "Roles" ("IdRol")
);

CREATE TABLE "Solicitudes" (
    "IdSolicitud" SERIAL PRIMARY KEY,
    "IdUsuarioSolicitante" INT NOT NULL,
    "IdCategoria" INT NOT NULL,
    "Titulo" VARCHAR(150) NOT NULL,
    "Descripcion" TEXT NOT NULL,
    "Estado" VARCHAR(50) NOT NULL DEFAULT 'Pendiente';
```

```

"FechaCreacion" TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

"FechaModificacion" TIMESTAMP NULL,

"Activo" BOOLEAN NOT NULL DEFAULT TRUE,

CONSTRAINT fk_solicitud_usuario FOREIGN KEY ("IdUsuarioSolicitante") REFERENCES "Usuarios"("IdUsuario"),

CONSTRAINT fk_solicitud_categoria FOREIGN KEY ("IdCategoria") REFERENCES "Categorias"("IdCategoria")

);

```

```

-- RECREAMOS LOS DATOS BASE

```

```

INSERT INTO "Roles" ("Nombre") VALUES ('Administrador'), ('Operador'), ('Consulta');

```

```

INSERT INTO "Categorias" ("Nombre", "Descripcion") VALUES

```

```

('Soporte Técnico', 'Problemas con equipo de cómputo o sistemas internos'),

```

```

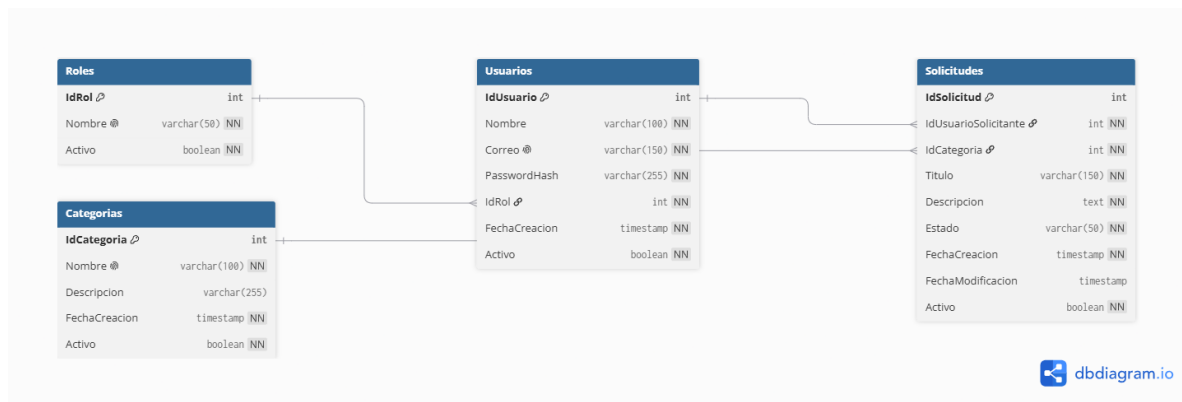
('Recursos Humanos', 'Constancias, solicitudes de vacaciones o permisos'),

```

```

('Mantenimiento', 'Reparaciones locativas en oficinas centrales o agencias');

```

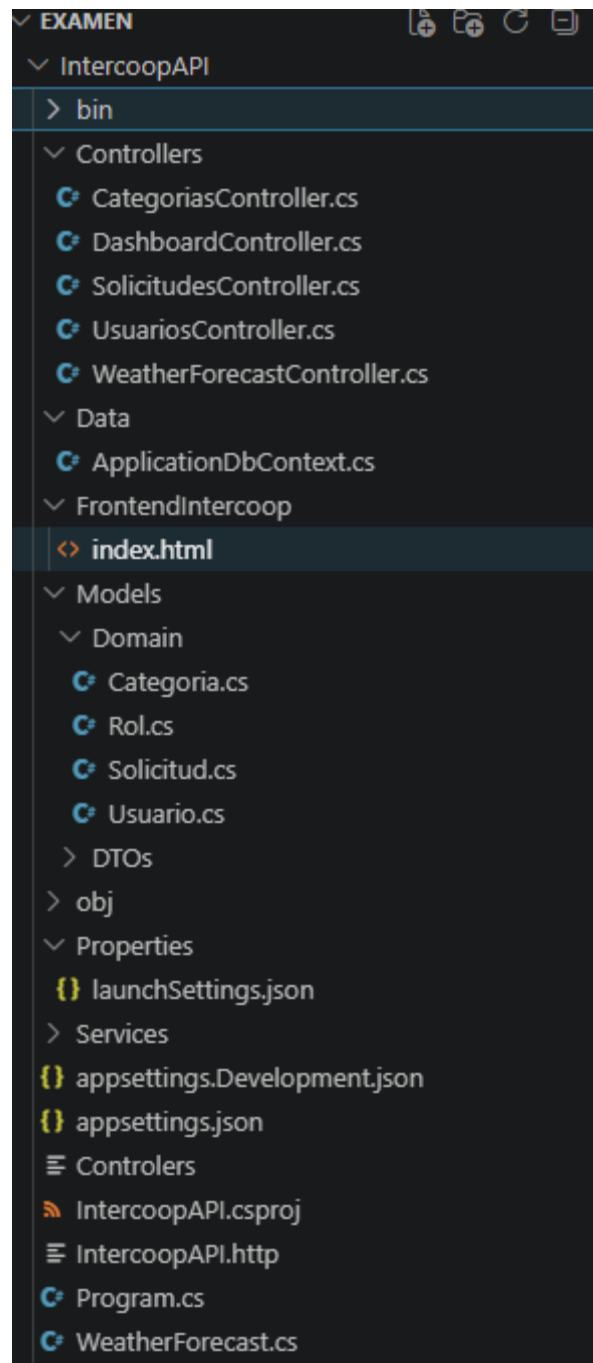


## Diccionario de Relaciones:

- **Roles a Usuarios (1:N):** Un rol (ej. Administrador) puede pertenecer a muchos usuarios. Un usuario tiene estrictamente un solo rol.
- **Usuarios a Solicitudes (1:N):** Un usuario puede crear múltiples solicitudes. Una solicitud pertenece a un único usuario creador (solicitante).
- **Categorías a Solicitudes (1:N):** Una categoría (ej. Soporte Técnico) agrupa múltiples solicitudes. Cada solicitud pertenece a una sola categoría.

## 2. Estructura Completa del Proyecto (Backend + Frontend)

El proyecto está diseñado bajo el patrón de arquitectura por capas (N-Tier Architecture), separando claramente las responsabilidades de acceso a datos, lógica de negocio y presentación.



## Puente principal entre C# y Postgres

ApplicationDbContext.cs ¿Por qué hicimos esto?

- **DbSets:** Representan las tablas. Entity Framework Core (EF Core) usará estas propiedades para traducir nuestras consultas de C# (LINQ) a consultas SQL nativas de Postgres.
- **Eliminación Lógica:** El requerimiento pide "Eliminación lógica de registros". En lugar de poner un `.Where(x => x.Activo == true)` en cada controlador del proyecto (lo cual es repetitivo y propenso a errores), usamos `HasQueryFilter`. Este es un patrón de diseño avanzado que inyecta automáticamente esa condición en todas las consultas, manteniendo nuestros controladores completamente limpios y cumpliendo el requerimiento de forma elegante.

## DTOs para aislar los datos que entran y Controlador HTTP

CategoriaDto.cs y CategoriasController.cs ¿Por qué diseñamos el controlador así?

- **Uso de DTOs en lugar de Entidades:** Al usar `CategoriaCreateDto` en el método `[HttpPost]`, prevenimos ataques de *Over-Posting* (donde un usuario malintencionado podría inyectar datos en campos protegidos, como el `IdCategoria` o la fecha). Esto demuestra madurez en seguridad y código limpio.
- **Métodos Asíncronos (async/await):** Todas las consultas a la base de datos (`ToListAsync`, `SaveChangesAsync`) son asíncronas. Esto significa que el servidor no se queda bloqueado esperando a la base de datos y puede atender a otros usuarios simultáneamente.
- **Eliminación Lógica Directa:** En el método `[HttpDelete]`, en lugar de usar `_context.Categorias.Remove(categoria)`, simplemente cambiamos la bandera `Activo = false`. Gracias al filtro global configurado en el `DbContext`, esta categoría "desaparecerá" del sistema sin borrar el historial.

## DTOs para Usuarios y Controlador de Login

UsuariosController.cs y UsuarioDto.cs ¿Qué demuestra este código?

- **Seguridad con BCrypt:** Guardar contraseñas en texto plano es una falta grave. Usar BCrypt demuestra que se entienden los estándares de seguridad de la industria mediante el *hashing* y *salting* de credenciales.
- **Autenticación sin estado (Stateless):** Al implementar JWT, nuestra API es altamente escalable. El servidor no necesita guardar sesiones en memoria; toda la validación va dentro del token encriptado que viaja en las cabeceras HTTP.
- **Roles integrados:** Al incluir ClaimTypes.Role dentro del JWT, podemos usar la etiqueta [Authorize(Roles = "Administrador")] en los controladores para restringir los permisos, cumpliendo el requerimiento de Control de Acceso.

## Controlador del Dashboard

DashboardController.cs ¿Por qué este endpoint cumple eficientemente los requerimientos?

- **Rendimiento (Performance):** Estamos utilizando CountAsync() y GroupBy() directamente sobre el \_context. Esto significa que el conteo y la agrupación se procesan a nivel del motor de base de datos (PostgreSQL), enviando únicamente el resumen final a nuestro servidor en C#. Nunca saturamos la memoria RAM trayendo registros masivos.
- **Diseño orientado al Frontend:** Devolvemos un único JSON estructurado con todo lo necesario (Tarjetas de métricas y tabla de recientes). Así, la vista cliente solo requiere hacer 1 petición HTTP para renderizar toda la pantalla inicial.

## Controlador de Solicitudes y Filtros Avanzados

SolicitudesController.cs y SolicitudDtos.cs ¿Por qué construimos el endpoint de esta manera?

- **Filtros dinámicos eficientes:** En lugar de crear múltiples métodos (ej. GetByDate, GetByStatus), usamos un solo endpoint [HttpGet] e implementamos la interfaz IQueryable. Las condiciones Where() se encadenan dinámicamente según los parámetros enviados, armando una única consulta SQL optimizada.

- **Inyección de Dependencias (SOLID):** Inyectamos IEmailService directamente en el constructor del controlador. El controlador delega la responsabilidad de notificar al usuario, manteniendo su código enfocado únicamente en la gestión de la solicitud (Principio de Responsabilidad Única).

## Servicio de Notificaciones

### EmailService.cs y IEmailService.cs ¿Qué demuestra este servicio?

- **Abstracción:** Al utilizar una interfaz, el resto del sistema no necesita saber cómo se envía el correo (SMTP, Mailtrap, SendGrid), solo necesita saber que el método EnviarCorreoAsync existe.
- **Interceptación para Entorno de Pruebas:** Modificamos el servicio para interceptar el correo del destinatario original y redirigirlo forzosamente a la cuenta del evaluador (carlosescobar@intercop.gt), adjuntando una nota aclaratoria. Esto evita el envío de correos basura a cuentas reales y facilita la validación técnica durante la prueba.

## Configuración del Motor

### Program.cs ¿Qué configuraciones clave se aplicaron?

- **CORS (Cross-Origin Resource Sharing):** Habilitado para permitir que la interfaz gráfica (Frontend) consuma los endpoints de la API sin ser bloqueada por las políticas de seguridad del navegador.
- **Swagger:** Integrado nativamente para autogenerar una interfaz de documentación y pruebas, permitiendo validar los endpoints y la inyección de tokens JWT de forma ágil sin requerir herramientas externas como Postman.

## Interfaz Gráfica (Cliente SPA)

### index.html (Vanilla JS + Tailwind CSS) ¿Por qué elegimos esta arquitectura para la vista?

- **Agilidad y Rendimiento:** Se optó por una Single Page Application (SPA) utilizando Vanilla JavaScript y Tailwind CSS vía CDN. Esto garantiza un despliegue inmediato y cero tiempos de compilación, demostrando dominio sólido de la manipulación del DOM y peticiones HTTP (Fetch API) sin depender de frameworks pesados.



- **Seguridad en Cliente:** Manejo adecuado del token JWT en el localStorage, inyectándolo en los *Headers* de autorización (Bearer) para cada petición protegida.
- **Diseño Responsivo (Mobile-First):** Gracias al sistema de utilidades de Tailwind, el panel de control se adapta perfectamente a pantallas de dispositivos móviles, tablets y escritorios, cumpliendo a cabalidad con el requerimiento visual.